

太原理工大学先进计算机系统实验室（ACSL）

适应期学员第二次学习路线

别样的Linux世界

难度指数：014

报酬：专业程序员基础（1）

要如何判断一个程序员是否专业？这是个很难有准确答案的问题——计算机涉及的领域太广了，一个程序员和另一个程序员专业的领域可能完全没有交集。但有一点可以确定，相比Windows，大部分专业程序员都更偏爱Linux。甚至微软自己也是为了Linux优秀的开发环境做了WSL（Windows Subsystem for Linux，Windows中的Linux子系统）。

本次作业提交要求跳转：[作业提交要求跳转链接](#)

迈向专业程序员的第二步——真正使用Linux

而经过上周的学习，想必你已经能够简单使用某个Linux发行版——Ubuntu。至少你已经知道如何打开终端并使用一些基础命令了。

但仅仅这种程度显然是远远不够的！

从现在开始，我们将会学习如何在Linux中编写源码、运行程序甚至管理自己的电脑！

命令行的艺术——其一

命令行的艺术（一）（必做）

命令行（CLI, Command-Line Interface），是程序员们喜爱Linux的最大原因之一。相比Windows命令行，Linux命令行更加美观、更加强大、更加便捷和自由，最重要的是——这是为所有Linux用户准备的！

学习以下两大内容（工具），掌握更多命令行相关操作。

1、APT包管理器

如果你在高考前就拥有了自己的电脑，那你应该不会对Windows上的软件安装感到陌生——毕竟只需要一直点确定嘛。如果你用得比较久，甚至接触了一些像“CMD”一样的东西，那你应该会经历

过很多**安装-卸载-安装-卸载**的循环，因为在Windows你所安装的东西都是打包好了的，要更换显然卸载重装要快，特别是下载了**盗版软件**的情况。

而在Linux你就无须担心，它有自己独特的包管理器——但请和软件商城这样的东西区分开，尽管现在它们看起来没什么不同。

而至于什么是包管理器？包就是电脑里的应用软件，那么包管理器就是电脑管理这些应用程序的工具。

Ubuntu中，最主流的包管理器是一个叫apt的东西，有时候也会是旧版本的apt-get。但是语法几乎都是一样的。

APT是Debian及其衍生（比如我们使用的Ubuntu就是）的**Linux软件包管理器**。APT可以自动下载，配置，安装二进制或者源代码格式的软件包，因而简化了Linux系统上管理软件的过程。

不过APT需要root权限，如果提示**Permission denied**，看看自己是不是没有**给权限（sudo）**。

常用命令

- 更新系统中的软件包列表： `apt update`
- 安装指定的软件命令： `apt install <package_name>`
- 安装多个软件包： `apt install <package_1> <package_2> <package_3>`
- 更新指定的软件命令： `apt update <package_name>`
- 删除软件包命令： `apt remove <package_name>`

使用技巧

- 快捷确认安装/删除： `apt install/remove <package_name> -y`
- 快速sudo：在Shell中， `!!` 表示上一条执行的命令，因此可以避免很多重复输入。

Permission denied example

```
1  $ apt install lolcat
2  E: Could not open lock file /var/lib/dpkg/lock-frontent - open (13:
   Permission denied)
3  E: Unable to acquire the dpkg frontend lock (/var/lib/dpkg/lock-frontent), are
   you root?
4  $ sudo !!                                # 此处 !! 会被替换为上文中的apt install lolcat
5  .....
6  [success!]
```

2、man (manual)

manual，也就是我们常说的官方手册，手册是汇集某一学科或某一主题等需要经常查考的资料，供读者随时翻检的工具书。手册主要为人们提供某一方面的基本规范，方便日常生活或学习。

我们不可能记住所有工具的使用细节，所以看手册是不可避免的。不过我们要怎么看手册呢？Linux有一个特殊的指令——man！

man man（必做）

使用 `man man` 命令，阅读其中的 `SYNOPSIS` 部分和 `DESCRIPTION` 部分，也就是语法和描述部分，学习有关man的一些知识。

你可以阅读 `man apt` 的语法页和描述页，看看有没有我们的讲义未提及的东西。如果你完成了拔高部分，你还可以阅读 `man vim`，`man readline`

tips: 全英阅读感到困难？或许你可以借助翻译软件。

中文输入法

用了好久的Linux，但仍然不知道怎么输入中文？尽管这不是必要的，但肯定有不少同学有这样的需求。

中文输入法

阅读链接内的教程完成中文输入法的配置：

<https://www.cnblogs.com/zensi/p/17725119.html>（在安装 `ibus-pinyin` 后如果没有出现图中的选项Chinese(Intelligent pinyin)，可尝试重启虚拟机）

如果以上链接中的方法不能正常使用中文输入法，可以试试这个 [📖 中文输入法安装教程](#)。

如果两种方法都不行，欢迎在群中提问。

在Linux中写C语言

从现在开始，我们将在Linux环境上进行我们的C语言编程了

在翁凯/慕课的C语言课程中，同学们都掌握了一部分C语言基础语法并学会使用了一个重要工具——Dev-cpp。但我们也说过Dev-cpp很快将被我们舍弃——就从现在开始！

VScode

VScode是微软旗下的一款代码编辑工具，其中集成了包括代码高亮、代码补全等在内的代码编辑辅助功能，相比Dev-cpp更为便捷、现代化。

阅读 [📖 Ubuntu安装VScode指西](#)

真正的编程环境配置

只在拥有了命令行后，环境的配置变得简单起来了。而现在的代码规模都不大，所以只需目前要用到两个工具——VScode和gcc。

gcc是一个编译器，能够将你的C语言源代码变成可执行文件。Dev-cpp中的一键编译运行就是依靠编译器完成的。为了继续深入学习计算机系统，我们将会把这一过程分离成单独的一个步骤。接下来就是gcc的学习。

未来，VScode的任务就是编辑代码，gcc的任务就是帮我们把代码编译成可执行文件。

！ 不知道如何用VScode编辑指定代码文件？

前面已经要求过你阅读 `code -help` 了，其中最开头的 `Usage:` 提示其实就告诉了你应该怎么做。

安装必要环境

Shell install gcc

```
1 $ sudo apt install gcc -y
```

gcc的简单使用

how to use gcc

```
1 $ gcc <source_code.c>      #不指定程序名，在当前目录下生成a.out程序
2 $ gcc <source_code.c> -o <program_name>    #指定程序名为program_name并生成
3 $ ./<program_name> #运行program_name
```

例如：

HALO

```
1 $ ls
2 hello.c
3 $ gcc hello.c -o halo
4 $ ls
5 hello.c halo
6 $ ./halo
7 Hello world!           # 程序halo (源码hello.c) 的运行输出
```

使用gcc编译并运行Hello world（必做）

Hello world伟大，无需多盐。

C语言学习（其二）—— 初窥门径

函数与数组

继续上次的C语言学习，直到完成**函数**和**数组**的学习。

以下是3种学习C语言的方式（视频，网站，书籍）：

视频：[翁恺C语言](#)，完成到80p数组的学习（翁恺老师课程时间极长，可根据自己理解情况适当开启倍速播放。如果无法适应这种倍速播放的学习方式，可以先尝试其他的学习方式）

书籍：群文件的【C语言相关】文件中由一些学习C语言的书籍，也可以阅读学习。

网站：下方的练习必做题

Tips: 推荐学习方式：**知识点+编程实践**

若选择群文件中的电子书，则需要学习到**函数**，**数组**知识点所在章节。

练习（必做）

仅仅学会了还不够，配上相应的练习才最佳。

在下面两个作业中**选择其中一个**作为本任务的练习对象（选择头歌的难度较大一些，但**黑客**们就是这样追求高难度问题的人）。

基础：慕课<https://www.imoooc.com/learn/249>（**对应范围：5-6章**）（比较简单，但是用于练手还是可以的，如果没有十足的把握就先来练练这个吧）

提高：头歌湖南工业大学C语言题目<https://www.educoder.net/paths/klbm7gto>（**第五部分到第六部分**）（——xpy学长碎碎念：难度高一些，但是这和咱们一些老师会安排的C语言课后作业大部分重复，所以学校的C语言课和作业会非常轻松）

其中包含了部分字符串（字符数组）的内容，但字符串的本质依旧是数组，完全可以仅用数组的处理方法来解决。

完成后将完成截图保存并重命名为 TheCoder 。

正确的编程（必做）

完成了函数与数组方面的学习和练习，你应该已经明白了为什么需要它们。现在，你需要将它们运用到你实现的终端（上周你用C语言完成的）里，**放到Linux环境下**。以及用**函数思想**修改你的代码，将你的两个模块以函数的方式实现在终端中。

不过，要是你还没写过注释的话，改代码会顺利吗？对于没有写代码注释的同学，建议直接在Linux按照函数思想重新写一遍。顺便写写**注释**吧，比如在函数旁批上其功能。

你可能会遇到一些小问题，这些问题在这个文档中能找到解法：[📖终端改进辅助说明](#)。

完成后将源代码重命名为 `Terminal2`。

作业提交

作业提交

1. 基础作业截图提交要求：创建一个新文件夹，命名格式为 `姓名-专业班级-Great-2` 并把截图放进去。
2. 基础作业源代码提交要求：将你的代码**源文件**也放在上述文件夹中。命名规则见任务布置处。
3. 将文件夹压缩为zip格式，保持压缩包名字与文件夹名字相同并提交至到 [第二周作业提交链接](#)
4. 如果你学有余力完成了下面的拔高**必做**内容，则把文件夹重命名，格式为 `姓名-专业班级-NewStar-2`。不要忘记更新你的源代码和截图，还有提交表单。

本周作业提交截止时间：**10月25号晚23:00**（如果因为自身原因未完成本周作业，在表单**请假说明**一栏填写原因，并写上你的预计提交时间即可）

任务链接指引：

必做：[链接1](#) [链接2](#)

拔高：[链接1](#) [链接2](#)

拔高内容

本次拔高内容主要在于学会写Shell脚本并练习Vim，其中Shell脚本的学习可能还需要你频繁得使用STFW，但完成后不仅能让你写出方便的Shell脚本，还能为你未来学习Makefile或者Python打下坚实基础。

命令行的艺术——其二

Vim

什么是Vim

Vim是一种编辑器，其特点是开源、高效，被称为**编辑器之神**。

编辑器是一种轻量级工具，主要用于编辑文本文件，例如代码文件或配置文件。它提供基本的文本编辑功能，如复制、粘贴、查找和替换等。编辑器通常不包含编译、调试等高级功能，适合简单的代码编写任务。常见的编辑器包括Vim和VS Code（未安装插件时）。

IDE则是一个功能全面的开发环境，集成了代码编辑、编译、调试、自动补全和代码分析等功能。它旨在为开发人员提供一站式的开发体验，支持从代码编写到测试的整个流程。

Vim高度的可扩展性，使其能通过插件和配置文件的相互组合，实现IDE的诸多功能：如代码补全，语法高亮等，然而对于大多数人来说，Vim有着一个比较陡峭的学习曲线。这意味着开始学习的时候可能会进展缓慢，不过一旦掌握一些基本操作之后，Vim就能帮你大幅提高编辑（写代码）效率。

如何使用Vim

Vim基础使用（拔高）

阅读下面这篇博客或该博客下提供的视频，掌握在终端使用Vim的方法。

<https://subingwen.cn/linux/vim/>

英语（Vim）练习（拔高）

Vim自带了一个练习教程，叫做vimtutor。你只需要在终端输入 `vimtutor` 命令即可进入。然后按照提示进行编辑吧，全部做完大概在30min左右。你需要至少完成前3章内容的练习（约10分钟）。

中文版vimtutor（支线）

你可能还是不愿意使用英文版的vimtutor，但为了必要的练习，你还是需要做vimtutor，不过是中文版的。这很简单，只需要输入 `vimtutor zh`。如果你发现它的内容是乱码，请联系群内助教。

但我们不推荐你使用中文版的vimtutor，vimtutor的英文难度并不高。不仅如此，你还需要先完成[中文输入法的安装](#)。

 一些公司会让所有员工使用相同的Vim配置，以便统一管理和使用。但是我们不要求大家这么做，我们鼓励大家自行探索适合自己的Vim配置。

? Vim因其开源特性，大部分Linux安装后都是自带Vim的。而开源带来的另一个好处就是安全。这也是许多公司喜爱Vim的原因之一。

? 另一个广被各大企业使用的编辑器是Emacs。其实，我们现在使用的终端就有Emacs的影子。保持对技术的热爱和好奇，你就能够在后面的讲义发现Emacs的存在。

默默奉献的阿卡林——GNU Readline

做完第一周的练习，想必你已经知道了自己制作的终端和Linux的终端究竟有多大的差距。最大的区别就是你无法通过键盘上的上下键切换历史命令！

回顾一下上一节讲义的内容，我们说操作系统也是程序。既然操作系统也是程序，那终端、命令行这些肯定也是程序，肯定能被人为实现。那么，这些操作是如何实现的呢？答案就是 `readline`！

不过理解readline对于现在的大家来说难度还是太高了，不符合我们循序渐进的教学理念。因此，现在只需要知道是谁在帮我们，又如何能在终端使用readline就足够了。

终端的readline操作

用过Vim后，如果你感觉对终端中的命令行进行编辑实在有点低效，那么学习readline操作就能让你使用终端更加得心应手。以下快捷键是较为常用的一些，推荐所有人掌握。

- `Ctrl + 左右方向键`：光标快速移动一个单词。
- `Ctrl + u`：删除光标前的所有字符。
- `Ctrl + k`：删除光标后的所有字符。
- `Ctrl + shift + -`：撤销上次更改。

如果你**已经熟悉**了Vim的操作，可以在终端使用 `set -o vi` 命令使用Vim模式的终端，但由于难以分辨当前所处模式，可能这不算是个好选择。往后我们介绍的也更多是Emacs模式的终端。

如果你想切换回默认的Emacs模式，使用 `set -o emacs`

? 你知道吗，由于Linux安全且自由的特性，大部分互联网公司都会选择Linux作为服务器的系统，也因此他们的员工也必须会使用Linux终端。而是否会使用readline，有时候就是分辨实习生和专业人员的一个显著区别。

? 想不到我们随手的操作，里面居然还隐藏了这么多东西！很多同学已经习惯了在终端里使用上下键进行历史命令切换，却对其如何实现全然不知，甚至在自己实现的简单终端里也下意识这样切换，以为这是理所当然的。不过至少现在我们知道了这是谁的付出——`readline`。你也明白了一点——计算机的学习要求我们把自己从这种理所当然中抽离出来。

不过，为什么会毫无察觉呢？

支线任务——计算机抽象

你突然产生了刚才的疑问，不知不觉就接触到了计算机领域中的一个重要概念——**抽象**。然而，抽象作为计算机领域中较难理解的一部分，同时也是被互联网增添了许多新意思的词语，对于大家来说可能有些难以理解。尝试利用上次讲义教给你的搜索技巧，通过**STFW**理解**抽象**的含义。

计算机教育中缺失的一课（节选）

Shell进阶

完成计算机教育中缺失的一课——[Shell 工具和脚本](#)的学习。这次可以不做课后习题，但是你需要完成以下任务。

你可能会用到这个作业帮助文档：[📖 Shell作业帮助文档](#)

编写一个shell脚本，能完成以下功能：

1. 在脚本文件所在目录创建一个 `build` 文件夹，若存在 `build` 文件夹则跳过这一步（不删除build文件夹）
2. 在build文件夹中创建一个文件，命名为 `log.txt`，若存在则跳过这一步（不删除log.txt文件）
3. 向log.txt中**追加写入**当前路径

在shell脚本最后用注释写明你完成了那些功能，例如：`# 完成1、2号功能，但3号没有完成`。

你也可以用英文注释，例如：`# complete : 1,2 incomplete : 3`

将你最终实现的脚本文件重命名命名为 `script.sh`，`cp` 复制到你的Windows共享文件夹中，随后转移到作业提交文件夹。

更多练习

我想要保存我的vimtutor进度？

写了好久的vimtutor退出后就没有了，感觉好空虚（？）其实vimtutor只是将某个文件复制并放在/tmp路径下然后再用vim打开，这个文件能在vim的左下角看见。退出后

vimtutor会自动清理这个文件，所以进度是无法保存的，这一点你可以从实践中得知，也能通过阅读vimtutor的源码得知。

而想要保留进度也很简单，只需要让这个复制的文件复制到我想要的地方，然后退出后不对其进行删除操作即可！有了思路，这就是一个新的Shell脚本练习了😊

你可以通过阅读 `help type` 或 `man which` 命令（它们都很短！），间接找到vimtutor的源码作为参考代码，甚至更改这个代码。最简单的方式就是删掉其中的“删除操作”代码。

完成后将你的代码文件cp到共享文件夹，重命名为 `new-vimtutor` 并转移到作业提交文件夹中。最后将文件夹压缩并提交到链接。

help or man? （很难）

奇怪，为什么 `type` 不能用 `man` 命令查找到手册呢？答案就在 `type type` 命令！（在终端试试吧）`type` 是bash的内置命令，其手册存在于 `man bash` 中，所以终端里想要获得 `type` 的 `man` 手册只能通过 `man bash` 查找。

但是能不能通过一些手段让我们在终端输入 `man type` 时也能直接获得文档，而不是发现报错后再用 `help type` 呢？毕竟想一开始就知道这条指令是不是内置的好像有点困难。

其实这也是个编写shell脚本的任务，但其涉及的知识已经超出了讲义需要大家掌握的知识范围了，也不会使大家的命令行更好用多少。如果你想要实现这个功能，可以自行探索更深入的Shell知识。

温馨提示

第二次学习路线到此结束，可别忘了提交作业到指定邮箱

本作品《"太理先进计算机系统研究实验室前置讲义适应期培养篇"》由 张勇俊 创作，并采用 CC BY-SA 4.0 协议进行授权。

遵循CC BY-SA 4.0开源协议：<https://creativecommons.org/licenses/by-nc-sa/4.0/deed.en>

转载或使用请标注所有者：张勇俊，太理先进计算机系统研究实验室